

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

OB 1조

김경민, 유재은, 전예원, 편한비

목차

- 논문 요약
- 실습
- 활용 및 개선 모델



1. 논문 요약

문제 정의 및 연구 배경

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

요약: 심층 합성곱 네트워크(ConvNet)에서 '네트워크의 깊이(Depth)'가 이미지 인식 정확도에 미치는 영향을 체계적으로 고찰한 연구

연구 배경

대규모 이미지/비디오 인식 분야에서의 ConvNet 성공 및 기존 AlexNet 구조 개선을 통한 정확도 향상 시도 지속

- 작은 수용장(Receptive Window) 및 스트라이드(Stride) 적용 (예: ILSVRC-2013 우승 모델)
- 다중 스케일 기반의 조밀한 모델 학습 및 테스트 방법 도입

문제 정의

기존의 다양한 하이퍼파라미터 튜닝 방식에서 탈피하여 아키텍처 설계의 핵심 요소인 '네트워크 깊이(Depth)'에 집중

- 연구 목적:**
- 기타 파라미터 고정 상태에서 레이어 적층 수 증가에 따른 효과를 엄밀하게 검증
- 깊은 아키텍처가 인식 정확도 개선에 기여하는 정도를 정량적으로 분석

핵심 아이디어

3x3 합성곱 필터의 전면 도입

대형 필터 대신 이미지의 좌/우, 상/하, 중심을 포착할 수 있는 가장 작은 크기인 3x3 필터만 연속으로 쌓아 성능을 극대화.

깊이와 파라미터 수의 이점



유효 수용장 유지

3x3 필터를 3개 쌓는 것은 단일 7x7 필터 레이어와 동일한 유효 수용장 (Effective Receptive Field)을 가짐.



비선형성 증가

레이어마다 활성화 함수(ReLU)가 추가되므로, 결정 함수(Decision Function)가 더욱 정교해지고 구별력이 높아짐.



파라미터 감소

7x7 필터 하나보다 3x3 필터 3개를 쓸 때 파라미터 수가 **약 81%** 감소하여 강력한 정규화 효과를 냄.

방법론

1. 입력 및 아키텍처 공통 레이어

- **입력:** 224x224 RGB 이미지 및 평균값 차감 전처리
- **합성곱:** 스트라이드 1, 패딩 1로 공간 해상도 보존
- **풀링:** 5개의 2x2 맥스 풀링 레이어 배치
- **FC 레이어:** 4096(2개) 및 1000(1개) 채널 구성
- **정규화:** 성능 향상 없는 LRN(AlexNet 방식) 제외

2. 훈련 및 테스트 기법

- **Scale Jittering:** [256, 512] 범위 무작위 샘플링 기반 훈련
- **Dense 평가:** FC를 합성곱으로 변환하여 전체 이미지 적용
- **Multi-crop 평가:** 3개 스케일 전반 총 150개 크롭 추출
- **결합 효과:** 두 방식의 상호 보완을 통한 최고 성능 달성

3. 실험 설정 (Setting)

- **데이터셋:** ILSVRC-2012 (훈련 130만, 검증 5만, 테스트 10만 장)
- **최적화:** 배치 256, 모멘텀 0.9, 가중치 감쇠 $5 \cdot 10^{-4}$, 드롭아웃 0.5
- **하드웨어:** NVIDIA Titan Black GPU 4대 동기화 방식 학습
- **비교 모델:** 11개(A)~19개(E) 레이어 기반 총 6개 구성 평가

실험 결과

1. ILSVRC 분류 결과

- **깊이의 효과:** 11층에서 19층으로 깊어질수록 에러율 일관적 감소
- **필터 크기 비교:** 3x3 필터(B)가 5x5 필터 대비 검증 에러 7%p 감소
- **단일 모델 성적:** 모델 E 기준 Top-5 검증 에러 7.5% 달성
- **앙상블 성적:** 2개 모델 결합 시 Top-5 테스트 에러 6.8% 기록

2. ILSVRC 국소화 결과

- **최적 설정:** 전 레이어 파인튜닝 및 클래스별 회귀(PCR) 적용 시 최상의 성적 도출
- **최종 성과:** 테스트 에러 25.3% 기록 및 **ILSVRC 2014 국소화 트랙 우승**

3. 범용성 검증

- **전이 학습 성능:** PASCAL VOC, Caltech 등 타 데이터셋에서도 특징 추출기로서 우수한 성능 발휘
- **경쟁력 입증:** 기존 최신 모델(SOTA)을 큰 폭으로 능가하며 압도적인 범용적 성능 증명

논문의 의의

- **'깊이'의 중요성 정립:** 복잡한 아키텍처 파라미터 변화 없이 오직 **'네트워크를 더 깊게 쌓는 것'**만으로도 시각 표상의 능력이 **비약적으로 상승함**을 깔끔하게 증명.
- **표준 가이드라인 제시:** 이후 딥러닝 연구에서 대형 필터(11x11, 7x7) 대신 **3x3 소형 필터를 겹쳐 쓰는 설계 방식**이 업계 **표준**으로 자리 잡도록 기여.
- **전이 학습(Transfer Learning)의 기반:** 본 연구를 통해 공개된 VGG-16, VGG-19 백본(Backbone) 네트워크는 객체 탐지, 세그멘테이션, 이미지 캡셔닝 등 컴퓨터 비전 전반의 베이스라인 특징 추출기로 오랫동안 활용.

한계 및 비판적 포인

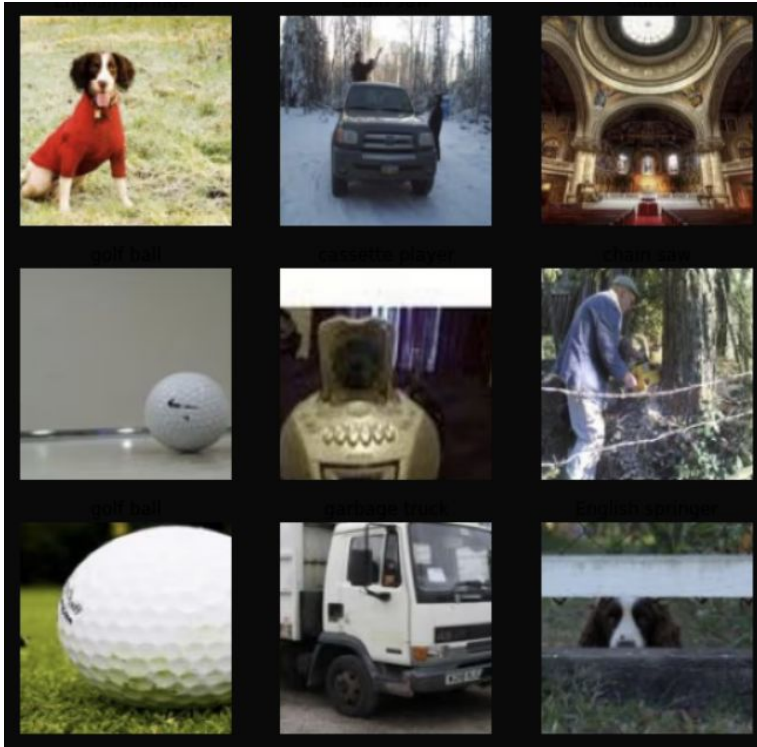
- **효율성(성능 포화)의 한계:** 레이어가 19층(E)에 도달했을 때 에러율 감소가 점차 수렴(Saturate)하는 모습을 보임. 이보다 더 깊은 층을 쌓기 위해서는 경사하강법 안정성이나 최적화 관점에서 또 다른 아키텍처적 돌파구 (예: 이후 등장한 ResNet의 잔차 연결 등)가 필요함을 시사.
- **대량의 파라미터와 연산 비용:** 소형 필터를 썼음에도 깊이가 깊어짐에 따라 모델 E의 파라미터 수는 **1억 4,400만 개**에 육박. 4대의 고성능 GPU로도 학습에 2~3주가 소요되는 만큼 , 동시대 경쟁 모델이었던 GoogLeNet(Szegedy et al., 2014)이 연산 효율성을 위해 복잡한 토폴로지를 썼던 것에 비하면 자원 소모가 크다는 단점 존재.



2. 실습

선정 데이터

- ImageNet160



- ImageNet에서 추출한 10개 클래스 분류 데이터셋
- Data split: Training set / Validation set
- Image type: 다양한 크기의 컬러 이미지
- Classes: tench, English springer, cassette player, chainsaw, church, French horn, garbage truck, gas pump, golf ball, parachute
- ImageNet보다 작아서 학습이 빠르고, 클래스가 명확해 모델 구조와 데이터 증강 효과 비교에 적합함

실험 1: 모델 깊이에 따른 이미지 분류 성능 비교

- VGG 계열 모델의 깊이가 깊어질수록 ImageNet 분류 성능과 학습 시간에 어떤 영향을 주는지 비교함

Model A	VGG11	가장 얇은 모델, 학습 속도 빠름
Model B	VGG16	중간 깊이, 성능과 시간 균형
Model C	VGG19	가장 깊은 모델, 표현력 높지만 학습 비용 증가

```
def get_vgg_model(model_name):
    if model_name == "vgg11":
        model = models.vgg11(weights=None)
    elif model_name == "vgg16":
        model = models.vgg16(weights=None)
    elif model_name == "vgg19":
        model = models.vgg19(weights=None)
    else:
        raise ValueError("model_name must be vgg11, vgg16, or vgg19")

    # ImageNet 1000-class classifier를 Imagenette 10-class로 변경
    model.classifier[6] = nn.Linear(4096, NUM_CLASSES)
    return model

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

실험 1: 모델 깊이에 따른 이미지 분류 성능 비교

- 고정조건

Optimizer	- SGD - Momentum = 0.9 - Weight Decay = 5e-4
Learning Rate	0.01로 시작함. 정확도가 향상되지 않을 때 10배씩 감소 (논문에서는 3회 감소)
Epoch	5 -> 10 -> 20 -> 30
Batch Size	64
Num_Classes	10
Image_size	224

- 측정지표

Test Accuracy	맞춘 개수 / 전체 개수
Training Time	에포크당 학습 시간 (GPU)
Parameter	파라미터 수
Loss Curve	모델 예측이 얼마나 틀렸는지

- Optimizer, learning rate, batch size, image size를 동일하게 설정하여 **모델 깊이 차이만 비교함**

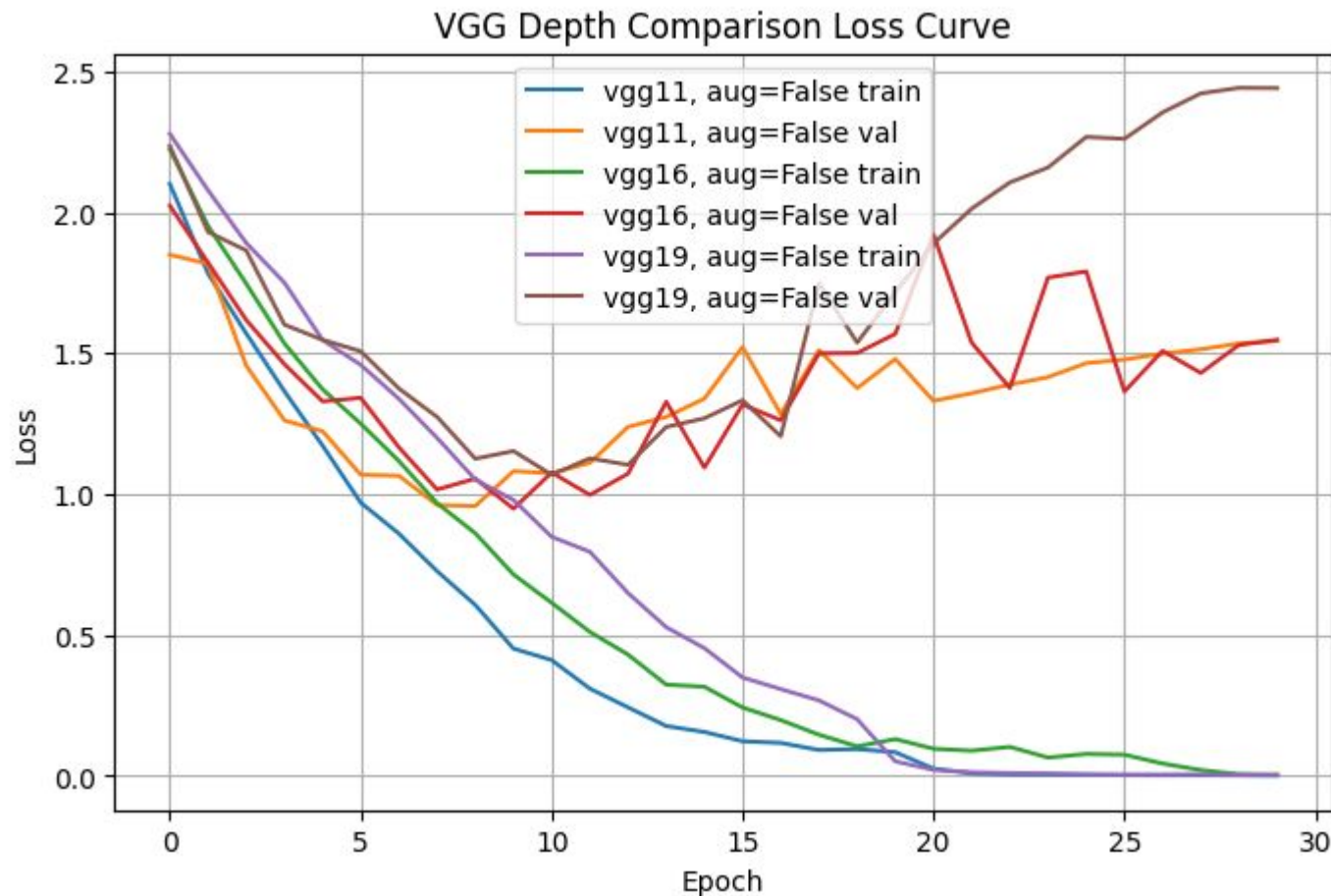
실험 1: 모델 깊이에 따른 이미지 분류 성능 비교

- Epoch 30 기준 모델 깊이 비교

	Model	Augmentation	Best Val Accuracy	Avg Epoch Time(sec)	Total Time(sec)	Parameters
0	vgg11	False	0.748535	93.512890	2805.433850	128807306
1	vgg16	False	0.715414	182.077669	5462.389199	134301514
2	vgg19	False	0.706242	214.075434	6422.341321	139611210

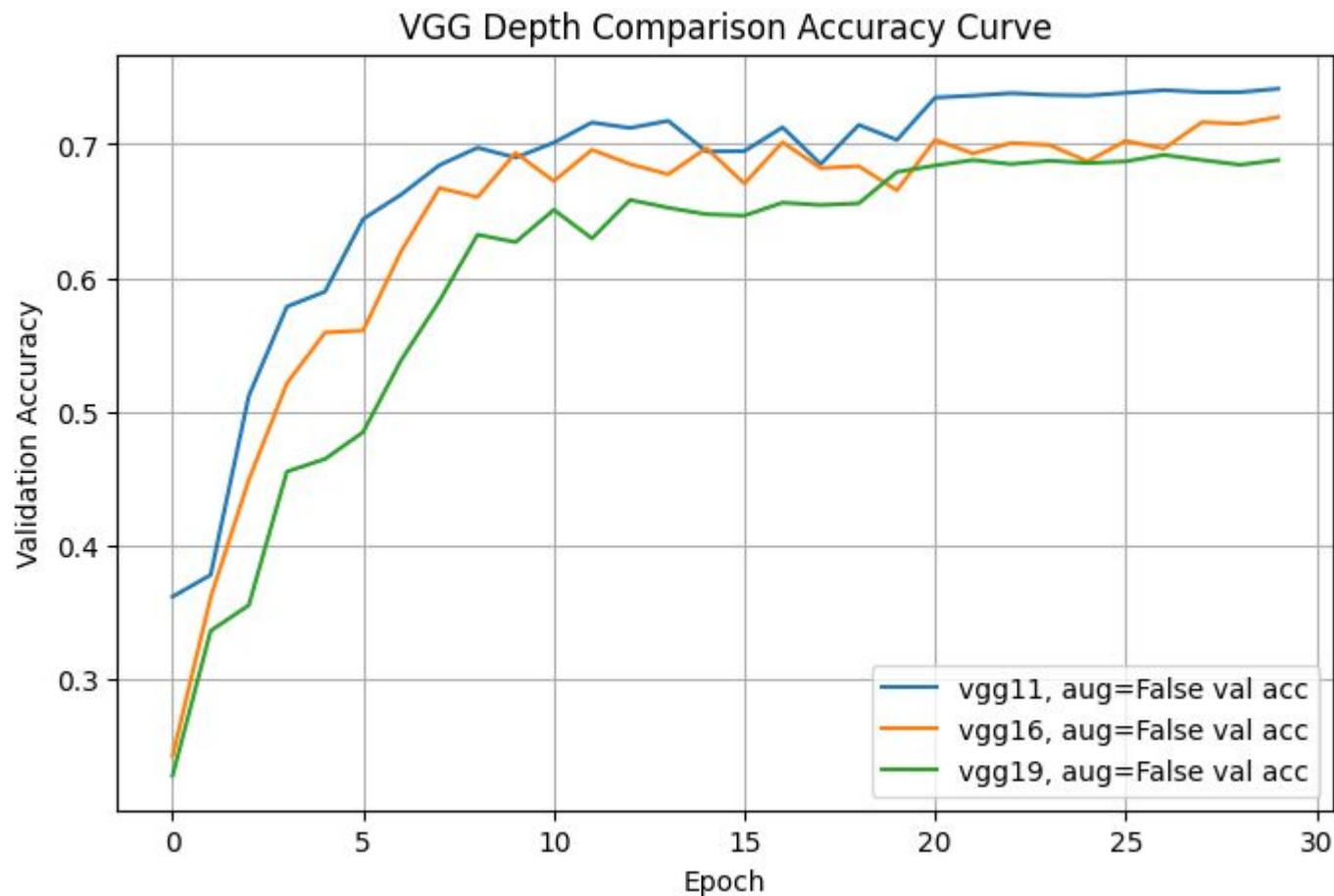
- VGG11: 가장 높은 validation accuracy를 보였고, 학습 시간도 가장 짧음
- VGG16 & VGG19: parameter 수와 학습 시간은 증가했지만, accuracy는 감소함

실험 1: 모델 깊이에 따른 이미지 분류 성능 비교



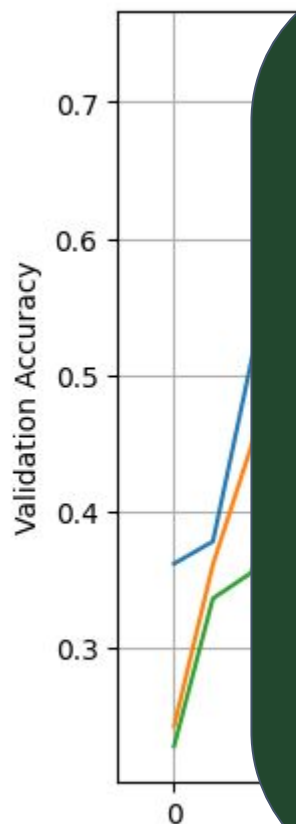
- VGG11
 - train loss와 val loss가 비교적 안정적으로 감소
 - 가장 안정적인 일반화 성능
 - VGG16 / VGG19
 - train loss는 계속 감소
 - val loss는 중반 이후 증가
 - overfitting 경향
- > 깊은 모델일수록 학습 데이터에는 잘 맞지만, validation 데이터에는 일반화가 잘 되지 않음
- 즉, overfitting 가능성이 높음

실험 1: 모델 깊이에 따른 이미지 분류 성능 비교



- VGG11
 - 가장 빠르게 accuracy 상승
 - 최종 accuracy도 가장 높음
 - VGG16 / VGG19
 - 초반에는 성능이 상승하지만
 - 후반부에는 accuracy가 정체됨
- > 깊은 모델이 항상 더 높은 정확도를 보장하지는 않음

실험 1: 모델 깊이에 따른 이미지 분류 성능 비교



한계점: 시간과 자원의 한계로 작은 데이터셋을 사용하여
논문과는 다른 결과가 나옴

-> 데이터셋 규모나 학습 epoch 수에 따라 더 단순한 모델이
더 효율적일 수 있음

-> 더 깊은 모델을 사용할 경우 충분한 데이터, 데이터 증강,
learning rate scheduling, early stopping 등을 함께
적용해야 일반화 성능을 높일 수 있음

성능

만
정체됨

정확도를

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석

데이터 증강(Data Augmentation)이란?

- 기존 학습 데이터를 변형하여 새로운 학습 데이터를 생성하는 기법
- 학습 데이터의 다양성을 증가시켜 모델의 일반화 성능 향상 기대

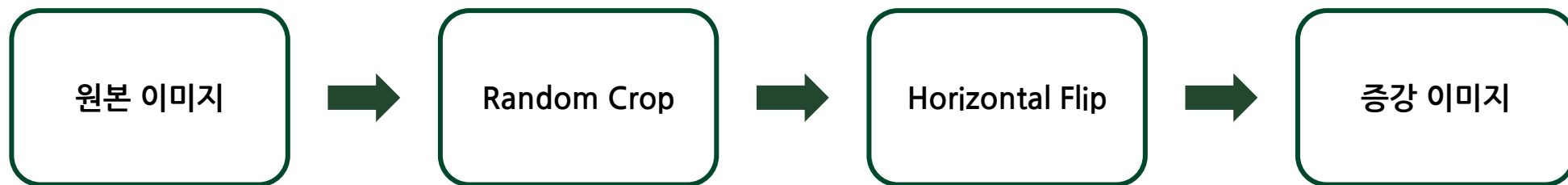
실험 목적

- 데이터 증강이 모델 성능에 미치는 영향 확인
- 데이터 증강이 과적합 완화에 효과적인지 확인

평가 방법

- Epoch별 Train Accuracy 측정
- Epoch별 Validation Accuracy 측정
- Train-Validation 성능 차이를 통해 과적합 여부 분석

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석



Random Crop

- 이미지를 무작위 위치에서 잘라 학습 데이터 다양성 증가

Horizontal Flip

- 이미지를 좌우 반전하여 다양한 시각적 패턴 학습

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석

Experimental Setup

항목	설정
Dataset	Imagenette
Model	VGG11
Epoch	30
Optimizer	SGD
Batch Size	64

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석

Experimental Setup

No Augmentation

- Resize(160x160)
- Normalize

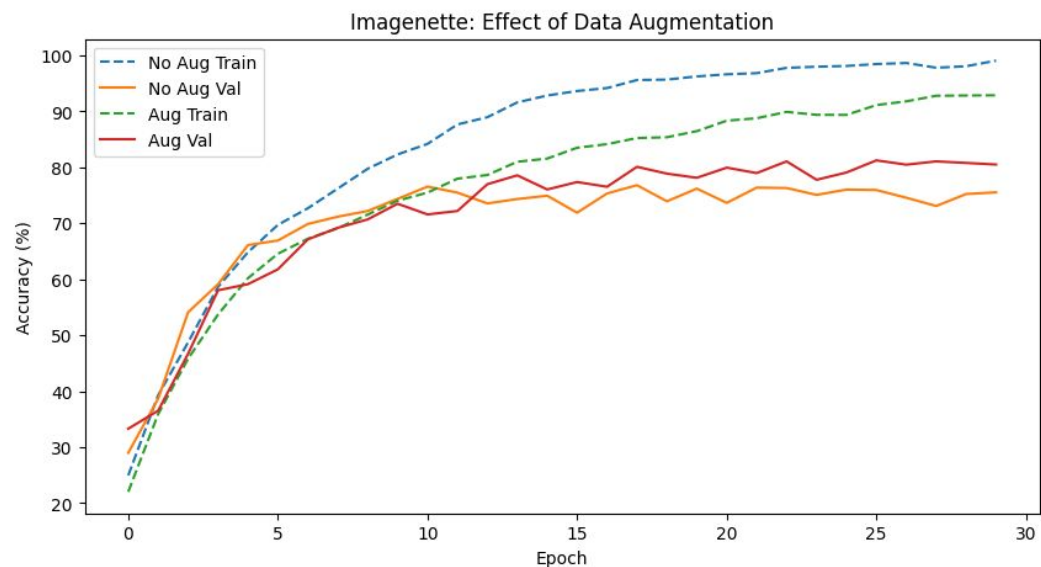
With Augmentation

- Resize(180x180)
- Random Crop(160x160)
- Random Horizontal Flip
- Normalize

=> 동일한 모델 및 학습 환경에서 데이터 증강 적용 여부만 비교

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석

Results



No Augmentation

- Train Accuracy가 빠르게 증가
- Validation Accuracy와의 격차 발생
- 과적합 경향 확인

With Augmentation

- Train Accuracy 상승 속도 완화
- Validation Accuracy 향상
- 일반화 성능 개선

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석

Conclusions

1. 데이터 증강은 학습 데이터의 다양성을 증가시킨다.
2. Validation 성능 향상에 긍정적인 영향을 미쳤다.
3. Train-Validation 성능 차이를 감소시켜 과적합을 완화하였다.
4. 데이터 증강은 모델의 일반화 성능 향상에 효과적임을 확인하였다.

실험 2: 데이터 증강의 일반화 및 과적합 방지 효과 분석

Limitations

- 제한된 증강 기법만 사용: 다양한 증강 기법의 효과는 분석하지 못함
- 단일 모델 사용: 다른 CNN 구조에서도 동일한 결과가 나타나는지 확인하지 못함
- 단일 데이터셋 사용: 다른 이미지 분류 데이터셋에 대한 일반화 가능성은 검증되지 않음

Future Work

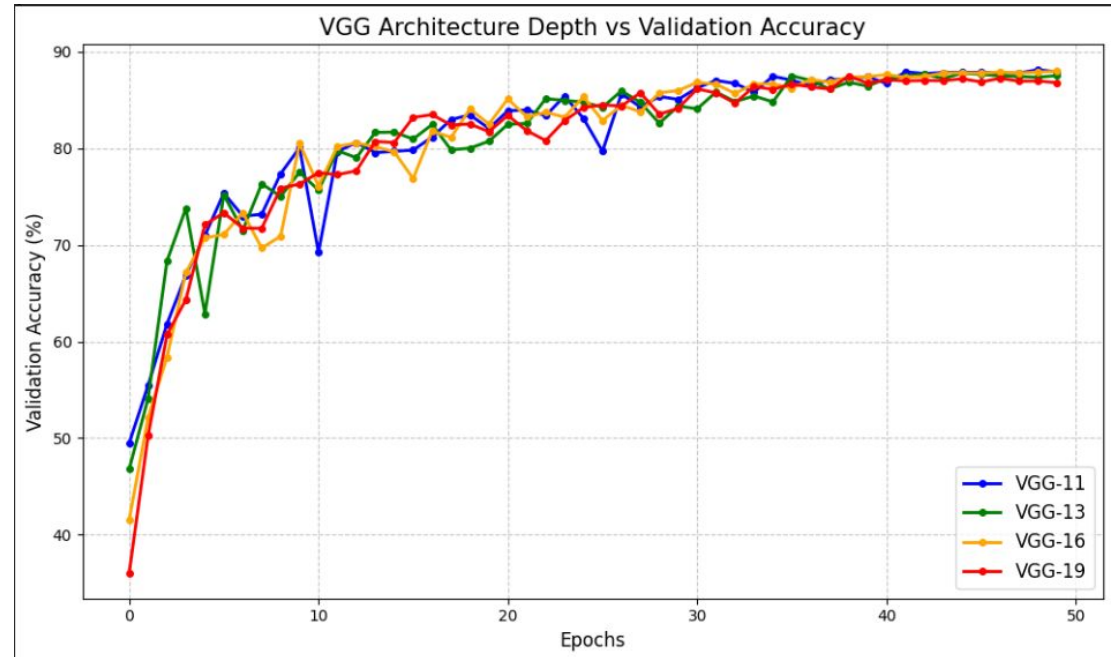
- 다양한 데이터 증강 기법에 대한 비교 실험 수행
- 여러 CNN 아키텍처에서의 일반화 성능 검증
- 데이터 증강 강도(예: Crop 비율, Flip 확률)에 따른 성능 변화 분석

추가 실험: VGG+배치정규화

[배치 정규화 도입]

배경 : VGG의 깊은 네트워크(11~19층) 학습 시 발생하는 학습 불안정성 및 기울기 소실 문제 해결 필요.

- ❑ **배치정규화의 역할** : 각 층의 입력 데이터 분포를 평균 0, 분산 1로 강제 조정하고 이를 통해 학습률을 높여도 안정적인 학습이 가능함.
- ❑ **성능 개선 효과** : 학습 수렴 속도가 향상되고 모델 깊이에 따른 성능 편차를 줄여 87~88% 수준의 안정적 정확도 확보
- ❑ **한계** : 근본적인 데이터셋 크기 부족으로 인한 과적합 자체를 해결하지 못함. (모델 복잡도와 데이터 규모의 불균형 문제 존재)

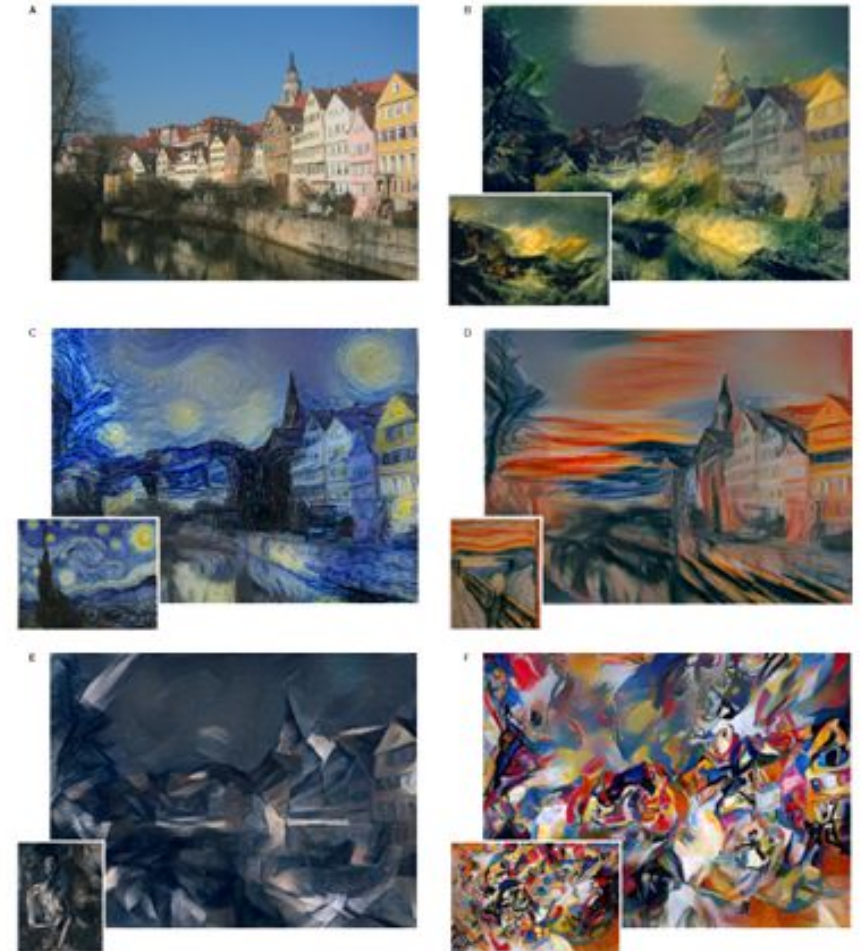


모델	최고 검증 정확도(%)	관찰 사항
VGG-11	88.15	가장 높은 정확도
VGG-13	87.80	성능 안정화 구간 도달
VGG-16	88	VGG-11과 유사한 수준
VGG-19	87.49	과적합 및 성능정체

VGG 활용 사례 - Neural Style Transfer

[예술적 이미지 생성]

- *A Neural Algorithm of Artistic Style* (Leon A. Gatys et al., 2016)
- ❑ **배경 지식** : 이미지의 내용(객체 형태)과 스타일 (화풍/질감)을 수학적으로 분리하여, 사진에 화가의 화풍을 입히는 기술.
- ❑ **활용 사례** : Prisma와 같은 예술 필터 앱 (사진을 고흐, 피카소 풍으로 변환), 디지털 콘텐츠 자동 생성 및 웹 디자인 자동화.
- ❑ **VGG 역할** : 이미지의 공간적 정보와 질감을 계층적으로 분리, skip connection 이 없는 합성곱 구조라 텍스처 정보 보존에 최적.

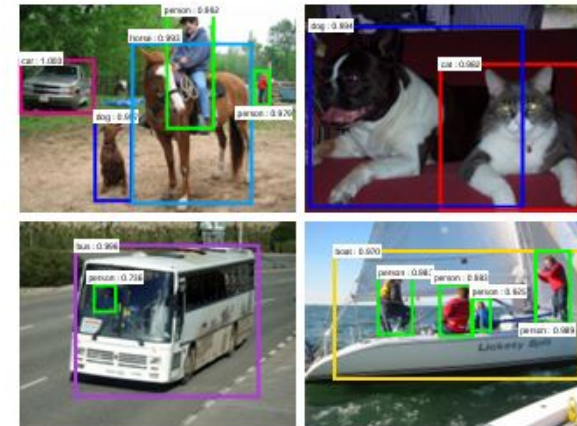
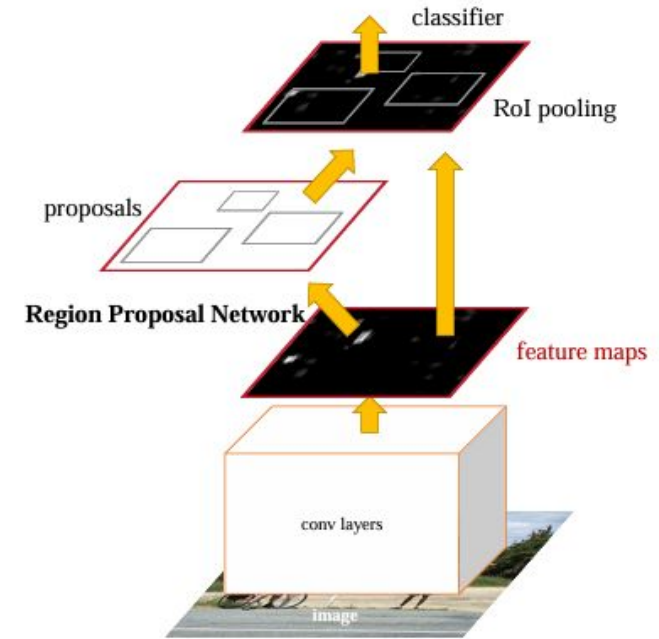


VGG 활용 사례 - Feature Extractor

[컴퓨터 비전의 기초 엔진]

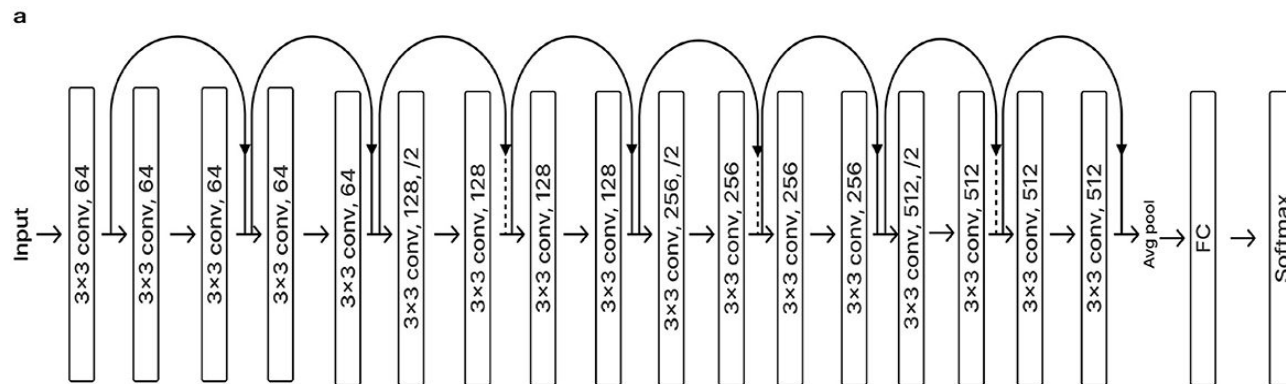
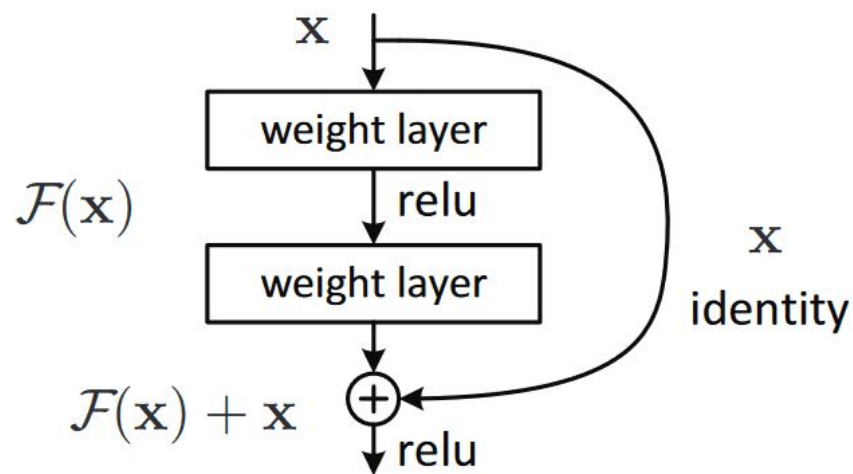
- *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks* (Ren et al., 2015)

- ❑ **배경 지식** : 이전의 객체 탐지 모델들은 영역을 찾는 알고리즘의 속도가 느리다는 점을 두고 Faster R-CNN은 이를 신경망 내에서 해결하고자 함.
- ❑ **활용 사례** : 자율주행 자동차의 보행자/신호등 탐지, 보안 CCTV의 침입자 자동 감지 시스템.
- ❑ **VGG 역할** : Faster R-CNN의 Backbone(특징 추출기)으로서, 입력된 이미지에서 저차원(윤곽선)부터 고차원(객체 파악) 특징 맵을 생성하여 Region Proposal Network에 전달함.
- ❑ **유용성** : VGG의 단순한 구조로 특징 맵을 예측 가능하게 생성. 따라서 탐지 알고리즘이 객체 위치를 잡을때 안정적인 베이스 라인이 됨



VGG 이후 개선 모델 - ResNet

- ❑ **알고리즘/ 원리** : Skip connection, 입력 데이터(x)를 층을 거친 결과($F(x)$)와 더해줌 ($F(x) + x$) -> 기울기 소실을 해결하기 위해 이전 층의 입력 정보를 연결하는 방법
- ❑ **개선점** : layer가 깊어질수록 성능이 떨어지는 문제 해결하여 수백 층까지 학습 가능하게 함.
- ❑ **전망** : 현대 컴퓨터 비전의 근간. 이후 등장하는 거의 모든 최신 모델의 설계의 기반(Skip Connection)으로 자리 잡음.



VGG 이후 개선 모델 - MobileNet

- ❑ **알고리즘/ 원리** : Depthwise Separable Convolution. 3x3 필터를 한 번에 계산하지 않고, 채널별로 나누어 계산 후 합치는 방식 (연산량 획기적 감소)
- ❑ **개선한 점**: 모델의 정확도 손실은 최소화하면서, 연산량과 파라미터 수를 대폭 줄임.
- ❑ **VGG의 한계 극복**: VGG의 단점인 너무 많은 파라미터 수와 높은 연산 비용 문제를 해결하여 모바일 환경에서도 구동이 가능하게 함.
- ❑ **추가 전망**: 온디바이스 AI(On-device AI) 시장의 확대에 의해, 앞으로도 효율적인 모델 설계의 표준이 될 것임.

* On-device : 데이터를 디바이스 자체에서 처리하는 방식

